Midas Vault Strategy Integration Audit

Côme du Crest

2026-03-26

Table of contents

- Table of contents
- Midas Vault Strategy Integration Audit
  - Scope
  - Context
  - Status
  - Legal Information And Disclaimer
- Issues
  - [Low] Rounding in RedemptionVaultWithMToken may revert

Midas Vault Strategy Integration Audit

This document presents the findings of a smart contract audit conducted by Côme du Crest.

For any inquiry contact me by email at come.ducrest@gmail.com or via telegram @ComeduCrest.

Scope

The scope includes all contracts of `midas-apps/contracts` modified in `PR 185`. At the time of review, the last commit of the PR is `0x67c35b9`.

The contracts have been updated to take into comments raised separately during the audit as well as the low severity issue and have been reviewed as of commit `0x62d441b`.

Context

The PR adds deposit and redeem vaults for Aave vaults, Morpho vaults, and mToken. A notion document has been given to describe the required the expected behaviour of the contracts (`here`).

The following contracts have been added/updated:

- RedemptionVaultWithAave: Redemption vault that withdraws aTokens from aave if necessary before redemption
- RedemptionVaultWithMorpho: Redemption vault that withdraws tokens from Morpho vaults if necessary before redemption
- RedemptionVaultWithMToken: Replaces some RedemptionVaultWithSwapper, storage layout is aligned. Previously used a liquidity provider to swap mToken1 to mToken2 and withdraw mToken2 from redemption vault to pay out user. Now directly uses underlying vault token held by the contract to get expected user tokens.
- RedemptionVaultWithSwapper: Fixed conversion when redeeming through a token swap where the amount of mToken redeemed on the underlying vault is rounded up to ensure enough tokens are received to fulfil the user redeem.
- MFOneRedemptionVaultWithMToken: Implements RedemptionVaultWithMToken with MFOne access control
- MSlRedemptionVaultWithMToken: Implements RedemptionVaultWithMToken with MSl access control
- DepositVault: Added a hook `_requestTransferTokensToTokensReceiver()` that is called in place of `_tokenTransferFromUser()` at the beginning of a deposit request. The default implementation is to transfer the deposit token without the fees to the vault token receiver. This does not change the behaviour for legacy vaults without auto-invest features.

- DepositVaultWithAave: Uses the added DepositVault hook to immediately deposit tokens to aave during instant deposits and deposit requests
- DepositVaultWithMorpho: Similar to DepositVaultWithAave
- DepositVaultWithMToken: Similar to DepositVaultWithAave

Risks of Aave and Morpho vaults

I assume that vaults with underlying Aave tokens do not use the deposited collateral to borrow other tokens from Aave. That would be the vault at risk of liquidation after redeems.

When redeeming with Aave or Morpho the mTokens redeemed get converted to USD and then to withdrawal tokens through one/two price feeds managed by Midas. If feeds have an outdated price this gives an arbitrage opportunity to Aave/Morpho token holders. This is an accepted risk by the protocol.

CPIMP

Contracts are deployed using a proxy pattern. The deploy script uses `openzeppelin/hardhat-upgrades` to deploy proxies that are initialized in the same transaction they are deployed. This prevents shenanigans in between deployment and initialization.

Status

The report has been sent to the core developer.

The low severity has been fixed as well as some comments raised separately and no issues remain.

Legal Information And Disclaimer

1. This report is based solely on the information provided by the protocol (the “Company”), with the assumption that the information provided is authentic, accurate, complete, and not misleading as of the date of this report. The auditor has not conducted any independent enquiries, investigations or due diligence in respect of the Company, its business or its operations.
2. Changes to the information contained in the documents, repositories and any other materials referenced in this report might affect or change the analysis and conclusions presented. The auditor is not responsible for monitoring, nor will we be aware of, any future additions, modifications, or deletions to the audited code. As such, the auditor does not assume any responsibility to update any information, content or data contained in this report following the date of its publication.
3. This report does not address, nor should it be interpreted as addressing, any regulatory, tax or legal matters, including but not limited to: tax treatment, tax consequences, levies, duties, data privacy, data protection laws, issues relating to the licensing of information technology, intellectual property, money laundering and countering the financing of terrorism, or any other legal restrictions or prohibitions. The auditor disclaims any liability for omissions or errors in the findings or conclusions presented in this report.
4. The views expressed in this report are solely my views regarding the specific issues discussed within this report. This report is not intended to be exhaustive, nor should it be construed as an assurance, guarantee or warranty that the code is free from bugs, vulnerabilities, defects or deficiencies. Different use cases may carry different risks, and integration with third-party applications may introduce additional risks.
5. This report is provided for informational purposes only and should not be used as the basis for making investment or financial decisions. This report does not constitute investment research and should not be viewed as an invitation, recommendation, solicitation or offer to subscribe for or purchase any securities, investments, products or services. The auditor is not a financial advisor, and this report does not constitute financial or investment advice.
6. The statements in this report should be considered as a whole, and no individual statement should be extracted or referenced independently.
7. To the fullest extent permitted by applicable laws, the auditor disclaims any and all other liability, whether in contract, tort, or otherwise, that may arise from this report or the use thereof.

Issues

[Low] Rounding in RedemptionVaultWithMToken may revert

Summary

The redemption vault with MToken attempts to withdraw an input quantity `amountTokenOut` of token from the underlying vault. It wants to ensure the withdrawal results in enough of tokens. To do that it uses a ceiling division to compute the amount of mToken to redeem, which will later be counter-balanced by a floor division in the underlying vault. However, when converting the missing amount to base 18 (`missingAmountBase18`) there is a floor division that could result in an insufficient amount of token withdrawn if the token uses more than 18 decimals causing a revert.

Vulnerability Detail

The function `_checkAndRedeemMToken()` converts the missing amount to base18 before rounding up the mTokeAAmount:

```
    function _checkAndRedeemMToken(
        address tokenOut,
        uint256 amountTokenOut,
        uint256 tokenOutRate
    ) internal {
        uint256 contractBalanceTokenOut = IERC20(tokenOut).balanceOf(
            address(this)
        );
        if (contractBalanceTokenOut >= amountTokenOut) return;

        uint256 missingAmount = amountTokenOut - contractBalanceTokenOut;
        uint256 tokenDecimals = _tokenDecimals(tokenOut);

        uint256 missingAmountBase18 = missingAmount.convertToBase18(
            tokenDecimals
        );  // @audit may round down if token decimals > 18
        uint256 mTokenARate = redemptionVault
            .mTokenDataFeed()
            .getDataInBase18();

        // Ceil so the inner vault's floored output is still >=
            missingAmountBase18.
        // Requires address(this) to have waivedFeeRestriction on the inner
            vault
        uint256 mTokenAAmount = Math.mulDiv(
            missingAmountBase18,
            tokenOutRate,
            mTokenARate,
            Math.Rounding.Up
        );
```

```
29
30          address mTokenA = address(redemptionVault.mToken());
31
32          require(
33              IERC20(mTokenA).balanceOf(address(this)) >= mTokenAAmount,
34              "RVMT: insufficient mToken balance"
35          );
36
37          IERC20(mTokenA).safeIncreaseAllowance(
38              address(redemptionVault),
39              mTokenAAmount
40          );
41
42          redemptionVault.redeemInstant(
43              tokenOut,
44              mTokenAAmount,
45              missingAmountBase18
46          );
47      }
```

Note that the conversion only rounds down if token is more than 18 decimals:

```
1       function convertToBase18(uint256 originalAmount, uint256 originalDecimals)
2           internal
3           pure
4           returns (uint256)
5       {
6           return convert(originalAmount, originalDecimals, 18);
7       }
8
9       function convert(
10          uint256 originalAmount,
11          uint256 originalDecimals,
12          uint256 decidedDecimals
13      ) internal pure returns (uint256) {
14          if (originalAmount == 0) return 0;
15          if (originalDecimals == decidedDecimals) return originalAmount;
16
17          uint256 adjustedAmount;
18
19          if (originalDecimals > decidedDecimals) {
20              adjustedAmount =
21                  originalAmount /
22                  (10**(originalDecimals - decidedDecimals));
23          } else {
24              adjustedAmount =
25                  originalAmount *
26                  (10**(decidedDecimals - originalDecimals));
27          }
28
29          return adjustedAmount;
30      }
```

Let's take the following numbers as an example:

```
assume amountTokenOut = 1e36
assume contractBalanceTokenOut = 1
then   missingAmount = 1e36 - 1 = 0.999...e36
then   missingAmountBase18 = 1e18 - 1 = 0.999...e18
assume tokenOutRate = 1
assume mTokenARate = 1
then   mTokenAAmount = missingAmountBase18
then   balanceAfterRedemption = 1e36 - 1e18 + 1 < amountTokenOut
```

The contract will not hold enough tokens to send to the user and the transaction will revert.

Impact

It is expected that the contract holds dust amounts of `tokenOut` due to the ceiling rounding in mToken to redeem. If output token has more than 18 decimals the redeem could revert due to floor rounding during conversion.

Code Snippets

https://github.com/midas-apps/contracts/blob/67c35b9423c1ed35363e3ab5360bb571d7d15a1f/contracts/RedemptionVaultWithMToken.sol#L222

https://github.com/midas-apps/contracts/blob/67c35b9423c1ed35363e3ab5360bb571d7d15a1f/contracts/libraries/DecimalsCorrectionLibrary.sol#L31

Recommendation

Use ceiling rounding when converting to base 18 in `_checkAndRedeemMToken`. Otherwise, convert the `tokenOut` balance to 18 decimals before doing the subtraction. This will work because `amountTokenOut` is already truncated to 18 decimals places by the function calling `_checkAndRedeemMToken()`.

Response

The codes has been fixed as of commit `0x62d441b` by following the recommendation and converting the `tokenOut` and contract balance to 18 decimals before doing the subtraction.